Multiple Monitors with Touchscreens

Rohit Goswami · 2020-07-11 · tools · workflow

> *A short tutorial post on multiple screens for laptops with touch-support and ArchLinux. Also evolved into a long rant, with an Easter egg.*

Background

Of late, I have been attempting to move away from paper, for environmental reasons[^fn:1]. Years of touch typing in Colemak (rationale, config changes) and a very customized Emacs setup (including mathematica, temporary latex templates, Nix, and org-roam annotations) have more or less kept me away from analog devices. One might even argue that my current website is essentially a set of tricks to move my life into `orgmode` completely.

However, there are still a few things I cannot do without a pointing device (and some kind of canvas). Scrawl squiggly lines on papers I'm reviewing. That and, scrawl weird symbols which don't actually follow a coherent mathematical notation but might be later written up in latex to prove a point. Also, and I haven't mastered any of the drawing systems (like Tikz) yet, so for squiggly charts I rely on Jamboard (while teaching) and Xournal++ for collaborations.

I also happen to have a ThinkPad X380 (try `sudo dmidecode -t system | grep Version`) which has an in-built stylus, and since Linux support for touchscreens from 2018 is known to be incredible, I coupled this with the ThinkVision M14 as a second screen.

X-Windows and X-ternal Screens

We will define two separate solutions:

mons
: Using arbitrary external monitors

autorandr
: Setting up profiles for specific monitors

Finally we will leverage both to ensure a constant touchscreen area.

Autorandr

I use the python rewrite simply because that's the one which is in the ArchLinux community repo. To be honest, I came across this before I (re-)discovered `mons`. The most relevant aspect of `autorandr` is using complicated configurations for different monitors, but it also does a mean job of running generic scripts as `postswitch` and `prefix` scripts.

Mons

`xrandr` is awesome. Unfortunately, more often than not, I forget the commands to interact with it appropriately. mons does my dirty work for me[^fn:2].

```
# -e is extend
mons -e left
# puts screen on the left
```

That and the similar `right` option, covers around 99% of all possible dual screen use-cases.

Constant Touch

The problem is that by default, the entire combined screen area is assumed to be touch-enabled, which essentially means an awkward area of the screen which is dead to all input (since it doesn't exist). The insight is that I never have more touch-enabled surfaces than my default screen, no matter how many extended screens are present. So the solution is:

> *Make sure the touch area is constant.*

We need to figure out what the touch input devices are:

```
xinput --list
```

```
⎡ Virtual core pointer                    id=2  [master pointer (3)]
⎜   ↳ Virtual core XTEST pointer              id=4  [slave pointer (2)]
⎜   ↳ Wacom Pen and multitouch sensor Finger touch id=10 [slave pointer (2)]
⎜   ↳ Wacom Pen and multitouch sensor Pen stylus  id=11 [slave pointer (2)]
⎜   ↳ ETPS/2 Elantech TrackPoint              id=14 [slave pointer (2)]
⎜   ↳ ETPS/2 Elantech Touchpad                id=15 [slave pointer (2)]
⎜   ↳ Wacom Pen and multitouch sensor Pen eraser  id=17 [slave pointer (2)]
⎣ Virtual core keyboard                   id=3  [master keyboard (2)]
    ↳ Virtual core XTEST keyboard             id=5  [slave keyboard (3)]
    ↳ Power Button                            id=6  [slave keyboard (3)]
    ↳ Video Bus                               id=7  [slave keyboard (3)]
    ↳ Power Button                            id=8  [slave keyboard (3)]
    ↳ Sleep Button                            id=9  [slave keyboard (3)]
    ↳ Integrated Camera: Integrated C         id=12 [slave keyboard (3)]
    ↳ AT Translated Set 2 keyboard            id=13 [slave keyboard (3)]
    ↳ ThinkPad Extra Buttons                  id=16 [slave keyboard (3)]
```

At this point we will leverage `autorandr` to ensure that these devices are mapped to the primary (touch-enabled) screen with a `postswitch` script. This `postswitch` script needs to be:

```
#!/bin/sh
# .config/autorandr/postswitch
xinput --map-to-output 'Wacom Pen and multitouch sensor Finger touch' eDP1
xinput --map-to-output 'Wacom Pen and multitouch sensor Pen stylus' eDP1
xinput --map-to-output 'Wacom Pen and multitouch sensor Pen eraser' eDP1
notify-send "Screen configuration changed"
```

The last line of course is really more of an informative boast.

At this stage, we have the ability to set the touchscreens up by informing `autorandr` that our configuration has changed, through the command line for example:

```
autorandr --change
```

Automatic Touch Configuration

Running a command manually on-change is the sort of thing which makes people think Linux is hard or un-intuitive. So we will instead make use of the incredibly powerful `systemd` framework for handling events.

Essentially, we combine our existing workflow with `autorandr-launcher` from here, and then set it all up as follows:

```
git clone https://github.com/smac89/autorandr-launcher.git
cd autorandr-launcher
sudo make install
sudo systemctl--user enable autorandr_launcher.service
```

Conclusion

We now have a setup which ensures that the touch enabled area is constant, without any explicit manual interventions for when devices are added or removed. There isn't much else to say about this workflow here. Additional screens can be configured from older laptops described here. Separate posts can deal with how exactly I meld Zotero (with WebDAV sync), `org-roam` and Xournal++ to wreak havoc on all kinds of documents. So, in-lieu of a conclusion, behold a recent scribble with this setup:

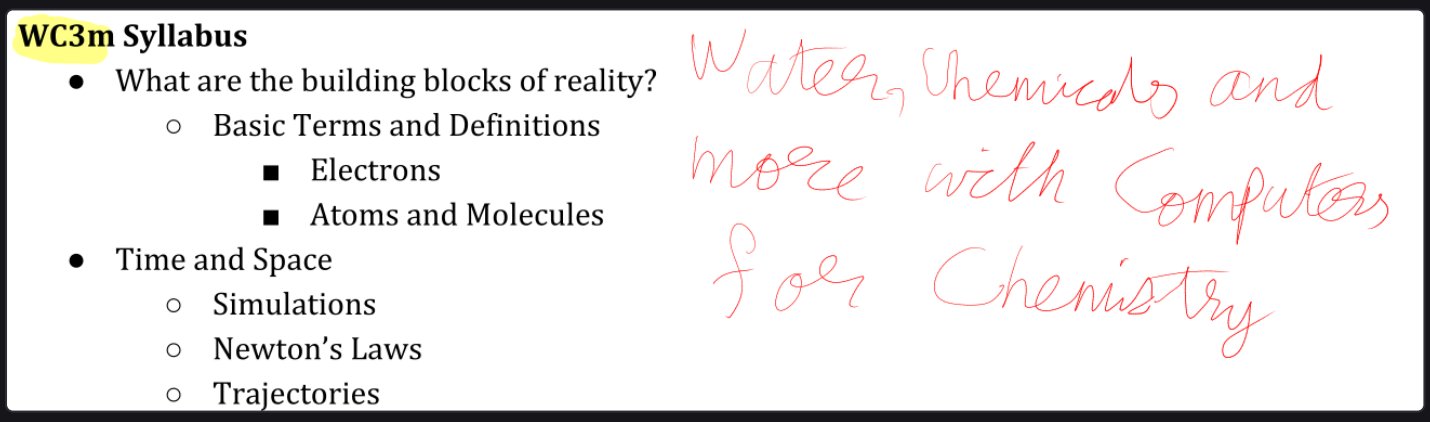

Figure 1: From the planning of [this voluntary course](http://www.wavelf.org/ij6TzydE3kTSF8cwwIUj)

[^fn:2]: I actually planned a whole post called "An Ode to Mons", when I first found out about it.